

# **System Design Document**

for

## **ADLynk Multi-Channel Advertising Management Platform**

### **Project Milestone 3**

Version 1.0

January 18, 2026

#### **Prepared by:**

Muhammad Hamza (NUM-BSCS-2024-48)  
Bilal Ahmed Khan (NUM-BSCS-2024-21)

Department of Computer Science  
Namal University, Mianwali

CSC-225 – Software Engineering

## Document Control

|                          |                                                                                                                       |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Document Title</b>    | System Design Document for ADLink Multi-Channel Advertising Management Platform                                       |
| <b>Project Name</b>      | ADLink - Multi-Channel Advertising Management Platform                                                                |
| <b>Version</b>           | 1.0                                                                                                                   |
| <b>Date</b>              | January 18, 2026                                                                                                      |
| <b>Status</b>            | Final                                                                                                                 |
| <b>GitHub Repository</b> | <a href="https://github.com/yourusername/ADLink">https://github.com/yourusername/ADLink</a>                           |
| <b>Figma Prototype</b>   | <a href="https://www.figma.com/prototype/your-prototype-link">https://www.figma.com/prototype/your-prototype-link</a> |
| <b>Meeting Minutes</b>   | <a href="https://docs.google.com/document/d/your-doc-link">https://docs.google.com/document/d/your-doc-link</a>       |

## Revision History

| <b>Date</b>      | <b>Version</b> | <b>Description</b>      | <b>Author</b>                    |
|------------------|----------------|-------------------------|----------------------------------|
| January 18, 2026 | 1.0            | Initial Design Document | Muhammad Hamza, Bilal Ahmed Khan |

# Contents

|          |                                              |           |
|----------|----------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                          | <b>5</b>  |
| 1.1      | Purpose . . . . .                            | 5         |
| 1.2      | Scope . . . . .                              | 5         |
| 1.3      | Document Organization . . . . .              | 5         |
| 1.4      | Design Methodology . . . . .                 | 6         |
| 1.5      | Reference Documents . . . . .                | 6         |
| <b>2</b> | <b>Design Assumptions and Constraints</b>    | <b>7</b>  |
| 2.1      | Design Assumptions . . . . .                 | 7         |
| 2.1.1    | User Environment Assumptions . . . . .       | 7         |
| 2.1.2    | Operational Assumptions . . . . .            | 7         |
| 2.2      | Design Constraints . . . . .                 | 7         |
| 2.2.1    | Platform Constraints . . . . .               | 7         |
| 2.2.2    | Performance Constraints . . . . .            | 8         |
| 2.2.3    | Security Constraints . . . . .               | 8         |
| 2.2.4    | Business Constraints . . . . .               | 8         |
| 2.2.5    | Technical Constraints . . . . .              | 9         |
| <b>3</b> | <b>Key Design Decisions</b>                  | <b>10</b> |
| 3.1      | Cross-Platform Mobile Architecture . . . . . | 10        |
| 3.2      | Firebase Backend-as-a-Service . . . . .      | 10        |
| 3.3      | Role-Based Access Control (RBAC) . . . . .   | 10        |
| 3.4      | Component-Based Architecture . . . . .       | 11        |
| 3.5      | RESTful API Design . . . . .                 | 11        |
| 3.6      | Offline-First Data Strategy . . . . .        | 11        |
| <b>4</b> | <b>Behavioral Diagrams</b>                   | <b>12</b> |
| 4.1      | Use Case Diagram . . . . .                   | 12        |
| 4.2      | Data Flow Diagrams . . . . .                 | 14        |
| 4.2.1    | Level 0 - Context Diagram . . . . .          | 14        |
| 4.2.2    | Level 1 DFD . . . . .                        | 15        |
| 4.2.3    | Level 2 DFD . . . . .                        | 16        |
| 4.3      | Sequence Diagram . . . . .                   | 17        |
| 4.3.1    | Service Provider Activity Diagram . . . . .  | 18        |
| 4.3.2    | Administrator Activity Diagram . . . . .     | 19        |
| <b>5</b> | <b>Structural Diagrams</b>                   | <b>21</b> |
| 5.1      | Class Diagram . . . . .                      | 21        |
| 5.2      | Component Diagram . . . . .                  | 22        |
| <b>6</b> | <b>Requirements-Design Traceability</b>      | <b>24</b> |
| <b>7</b> | <b>Project Resources</b>                     | <b>26</b> |
| 7.1      | GitHub Repository . . . . .                  | 26        |
| 7.2      | Figma Prototype . . . . .                    | 26        |
| 7.3      | Meeting Minutes . . . . .                    | 26        |

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| <b>8</b> | <b>Meeting Summary</b>                             | <b>27</b> |
| 8.1      | Meeting Overview . . . . .                         | 27        |
| 8.2      | First Meeting - Design Review . . . . .            | 27        |
| 8.2.1    | Key Discussion Points . . . . .                    | 27        |
| 8.2.2    | Feedback Received . . . . .                        | 27        |
| 8.2.3    | Action Items . . . . .                             | 27        |
| 8.3      | Second Meeting - Prototype Demonstration . . . . . | 28        |
| 8.3.1    | Key Discussion Points . . . . .                    | 28        |
| 8.3.2    | Feedback Received . . . . .                        | 28        |
| 8.3.3    | Approval Status . . . . .                          | 28        |
| 8.4      | Design Validation . . . . .                        | 29        |
| <b>9</b> | <b>Conclusion</b>                                  | <b>30</b> |

## List of Figures

|   |                                                         |    |
|---|---------------------------------------------------------|----|
| 1 | Use Case Diagram . . . . .                              | 13 |
| 2 | Data Flow Diagram - Level 0 (Context Diagram) . . . . . | 14 |
| 3 | Data Flow Diagram - Level 1 . . . . .                   | 15 |
| 4 | Data Flow Diagram - Level 2 . . . . .                   | 31 |
| 5 | Sequence Diagram . . . . .                              | 32 |
| 6 | Activity Diagram - Business Owner . . . . .             | 32 |
| 7 | Activity Diagram - Administrator . . . . .              | 33 |
| 8 | Class Diagram . . . . .                                 | 34 |
| 9 | Component Diagram . . . . .                             | 35 |

# 1 Introduction

## 1.1 Purpose

This System Design Document provides a comprehensive design specification for the ADLink Multi-Channel Advertising Management Platform. The document translates the requirements specified in the approved Software Requirements Specification (SRS) into a detailed system design that serves as a blueprint for implementation.

This document is intended for:

- Development team members who will implement the system
- System architects who need to understand the overall system structure
- Quality assurance team for validation and verification
- Project stakeholders for design review and approval
- Requirement provider (Saqlain Baig) for validation
- Academic supervisors and course instructors

## 1.2 Scope

The design encompasses all aspects of the ADLink system including:

- Mobile application architecture design for Android and iOS platforms
- Behavioral models showing system dynamics and user workflows
- Structural models depicting system components and their relationships
- User interface design through interactive prototypes
- Backend system architecture and database design
- Complete traceability from requirements to design elements
- Integration patterns for third-party services (payment gateways, maps, notifications)

## 1.3 Document Organization

This document is organized into the following sections:

- **Section 1:** Introduction - Purpose, scope, and design methodology
- **Section 2:** Design Assumptions and Constraints - Environmental and technical limitations
- **Section 3:** Key Design Decisions - Architectural choices and rationale
- **Section 4:** Behavioral Diagrams - Use cases, data flow, sequence, and activity diagrams

- **Section 5:** Structural Diagrams - Class and component diagrams
- **Section 6:** Requirements-Design Traceability - Mapping requirements to design
- **Section 7:** Project Resources - Repository and prototype links
- **Section 8:** Meeting Summary - Stakeholder feedback and approvals
- **Section 9:** Conclusion - Summary and next steps

## 1.4 Design Methodology

The system design follows object-oriented design principles and mobile-first architecture patterns using standard UML notation. The design process involved:

1. **Requirements Analysis** - Review of approved SRS document
2. **Architecture Planning** - Selection of cross-platform mobile framework and back-end technology
3. **Behavioral Modeling** - Creation of use case, data flow, sequence, and activity diagrams
4. **Structural Modeling** - Definition of system structure through class and component diagrams
5. **Prototype Development** - Creation of interactive Figma prototype for mobile interfaces
6. **Stakeholder Validation** - Review meetings with Requirement Provider (Saqlain Baig)
7. **Traceability Analysis** - Mapping of functional requirements to design elements

## 1.5 Reference Documents

1. Software Requirements Specification (SRS) for ADLink, Version 1.0, November 28, 2025
2. IEEE Std 830-1984, IEEE Guide to Software Requirements Specifications
3. UML 2.5 Specification
4. Flutter Design Patterns and Best Practices
5. Firebase Architecture Guide
6. Mobile App Design Guidelines (iOS Human Interface Guidelines, Material Design)
7. ADLink Project Proposal, November 2025

## 2 Design Assumptions and Constraints

### 2.1 Design Assumptions

#### 2.1.1 User Environment Assumptions

1. Users have access to smartphones running Android 8.0+ or iOS 12.0+
2. Users have reliable mobile internet access (3G/4G/WiFi) with minimum 1 Mbps bandwidth
3. Four distinct user roles are sufficient: Business Owners, Service Providers, Asset Owners, and Administrators
4. Users possess basic smartphone operation skills and can navigate mobile applications
5. Most users have devices with camera capability for uploading images and media
6. Users are comfortable with digital payment methods (mobile wallets, cards)
7. Service providers and asset owners will maintain accurate and updated listings

#### 2.1.2 Operational Assumptions

1. Sufficient service providers will register to offer diverse advertising options
2. At least one system administrator will be available for platform management
3. System will handle average of 100-200 bookings per day
4. Peak usage will not exceed 1000 concurrent users
5. Database will grow approximately 50,000 records per year
6. Payment gateway services will remain operational and affordable
7. Third-party APIs (maps, notifications) will maintain acceptable uptime

### 2.2 Design Constraints

#### 2.2.1 Platform Constraints

1. System must be a cross-platform mobile application (Android and iOS)
2. Must support Android 8.0 (API Level 26) and above
3. Must support iOS 12.0 and above
4. Application size must not exceed 100 MB for app store compliance
5. Must not require device rooting or jailbreaking
6. System requires continuous internet connectivity for core features
7. Must comply with Google Play Store and Apple App Store guidelines

### **2.2.2 Performance Constraints**

1. Application launch time must not exceed 3 seconds
2. User action response time must be within 1 second for 95% of operations
3. System must support minimum 1000 concurrent users
4. Maximum image upload size: 5 MB per file
5. API response time must not exceed 2 seconds
6. Real-time notifications must be delivered within 5 seconds
7. Database queries must execute within 500ms for standard operations

### **2.2.3 Security Constraints**

1. All data transmission must use HTTPS/TLS 1.2 or higher
2. Passwords must meet complexity requirements (8+ characters, uppercase, lowercase, number, special character)
3. User sessions must expire after 24 hours or 30 minutes of inactivity
4. Payment information must never be stored locally on devices
5. Role-based access control must be strictly enforced
6. All sensitive data must be encrypted at rest and in transit
7. Must comply with Pakistan's data protection regulations

### **2.2.4 Business Constraints**

1. Project operates under student project budget constraints
2. Development must align with academic semester schedule (November 2025 - March 2026)
3. Team limited to 2 developers
4. Must deliver MVP within semester timeframe
5. Third-party service costs must be minimized (use free tiers where possible)
6. Design must be approved by Requirement Provider (Saqlain Baig)

### **2.2.5 Technical Constraints**

1. Must use approved technology stack (Flutter/React Native, Firebase)
2. Backend limited to Node.js or Python (Django/Flask)
3. Database limited to Firebase Realtime Database or Firestore
4. Payment gateway integration limited to available APIs in Pakistan
5. Map services limited to Google Maps or equivalent free alternatives
6. Limited third-party API integrations due to cost constraints

## 3 Key Design Decisions

### 3.1 Cross-Platform Mobile Architecture

**Decision:** Adopt Flutter framework for cross-platform mobile development with a single codebase for Android and iOS.

**Rationale:**

- Single codebase reduces development time and maintenance effort
- Native-like performance on both platforms
- Rich UI component library for consistent user experience
- Strong community support and extensive documentation
- Cost-effective for student project with limited resources
- Faster iteration and prototyping capabilities

### 3.2 Firebase Backend-as-a-Service

**Decision:** Use Firebase as the primary backend infrastructure including Realtime Database/Firestore, Authentication, Cloud Messaging, and Cloud Storage.

**Rationale:**

- Reduces backend development complexity
- Provides real-time data synchronization out-of-the-box
- Built-in authentication and security rules
- Scalable infrastructure without server management
- Free tier sufficient for MVP and testing
- Seamless integration with Flutter
- Built-in analytics and crash reporting

### 3.3 Role-Based Access Control (RBAC)

**Decision:** Implement comprehensive role-based access control with four primary user roles: Business Owners, Service Providers, Asset Owners, and Administrators.

**Rationale:**

- Different users have fundamentally different needs and permissions
- Ensures security and data privacy across user types
- Simplifies permission management through role hierarchies
- Aligns with business model and organizational structure
- Prevents unauthorized access to sensitive features
- Supports future role additions and modifications

### 3.4 Component-Based Architecture

**Decision:** Organize the mobile application into modular, reusable components following Flutter’s widget-based architecture and MVVM (Model-View-ViewModel) pattern.

**Rationale:**

- Clear separation of concerns between UI, business logic, and data
- Components can be developed and tested independently
- Promotes code reusability across different screens
- Easier to maintain and extend functionality
- Supports parallel development by team members
- Facilitates unit and integration testing

### 3.5 RESTful API Design

**Decision:** Design backend APIs following RESTful principles with JSON data format for client-server communication.

**Rationale:**

- Industry-standard approach for mobile-backend communication
- Stateless architecture improves scalability
- Clear and intuitive endpoint structure
- Easy to document and test
- Wide tooling and library support
- Enables future integration with web or other platforms

### 3.6 Offline-First Data Strategy

**Decision:** Implement local caching with offline capabilities for viewing previously loaded data.

**Rationale:**

- Improves user experience in areas with poor connectivity
- Reduces API calls and bandwidth usage
- Faster app performance through local data access
- Critical for mobile users in Pakistan with variable network quality
- Firebase provides built-in offline persistence
- Seamless synchronization when connectivity restored

## 4 Behavioral Diagrams

### 4.1 Use Case Diagram

The Use Case Diagram illustrates the primary interactions between different actors and the ADLink system. The diagram identifies four main actors: Business Owners, Service Providers, Asset Owners, and System Administrators, each with their specific use cases.

**Key Use Cases:**

**Business Owner Use Cases:**

- Register and authenticate
- Browse advertising services
- Search and filter services
- View service provider profiles
- Create advertising campaigns
- Book services and assets
- Process payments
- Track campaign performance
- Rate and review services

**Service Provider Use Cases:**

- Register and set up profile
- List advertising services
- Manage service pricing and availability
- Receive and manage bookings
- Communicate with clients
- Track revenue and analytics

**Asset Owner Use Cases:**

- Register assets (billboards, vehicles)
- Update asset availability
- Manage asset pricing
- Process booking requests
- Track asset utilization

**Administrator Use Cases:**

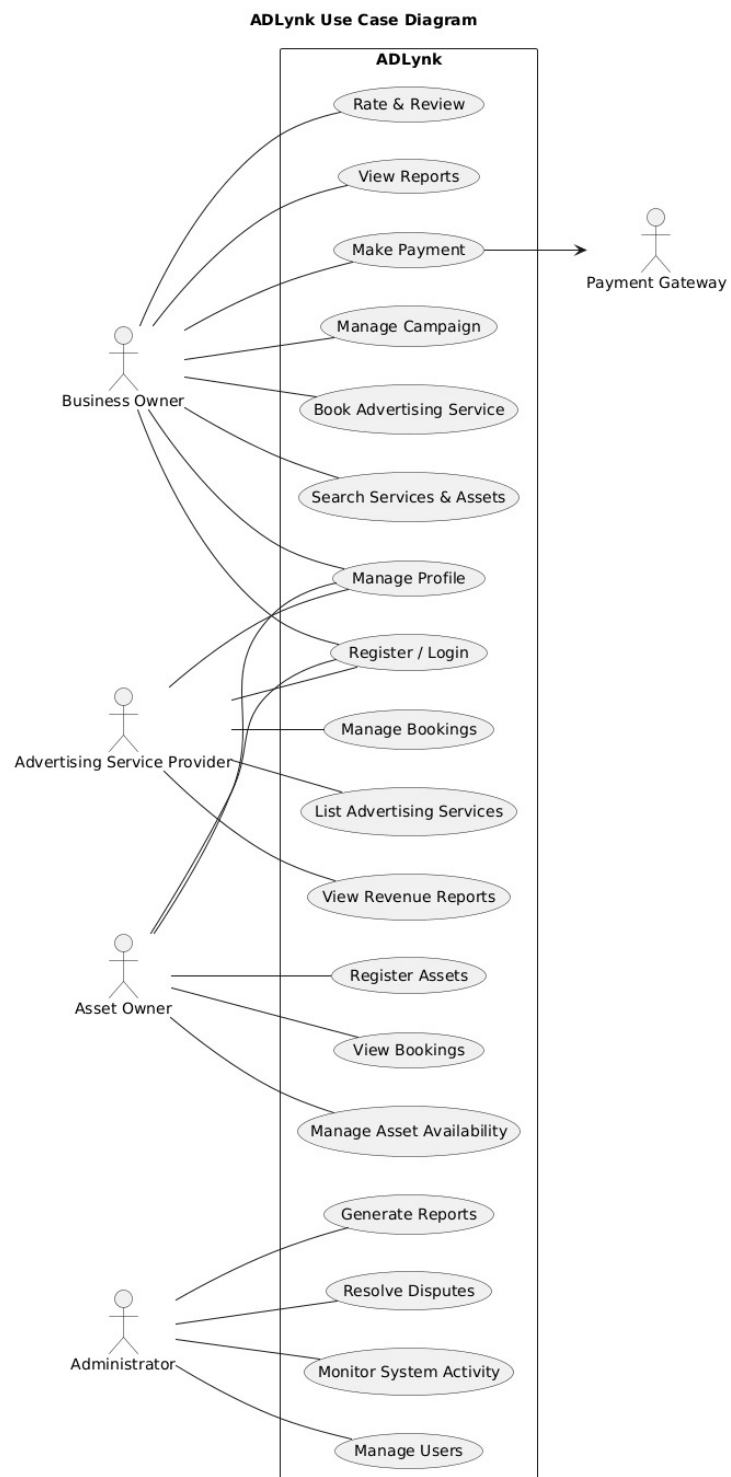


Figure 1: Use Case Diagram

- Manage user accounts
- Monitor platform activity
- Resolve disputes
- Generate system reports
- Configure system settings

## 4.2 Data Flow Diagrams

### 4.2.1 Level 0 - Context Diagram

The Context Diagram shows the ADLink system boundary and its interactions with external entities including Business Owners, Service Providers, Asset Owners, Administrators, Payment Gateway, Map Services, and Notification Services.

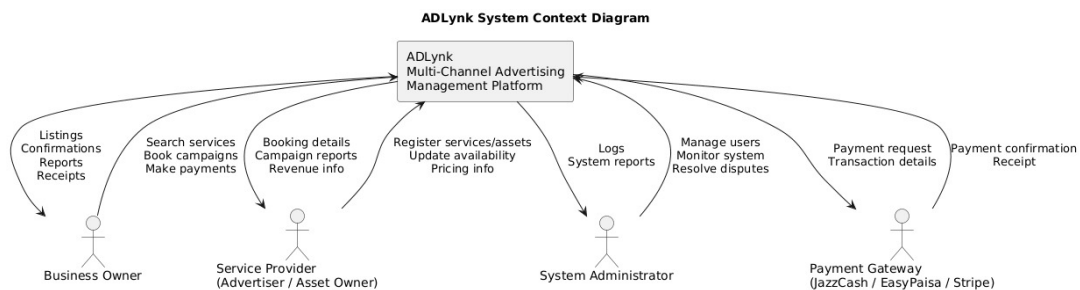


Figure 2: Data Flow Diagram - Level 0 (Context Diagram)

#### External Entities and Data Flows:

##### Business Owners ↔ ADLink System:

- Input: Search queries, booking requests, campaign data, payment information
- Output: Service listings, booking confirmations, campaign analytics, receipts

##### Service Providers/Asset Owners ↔ ADLink System:

- Input: Service registrations, availability updates, pricing information
- Output: Booking notifications, client information, revenue reports

##### Administrators ↔ ADLink System:

- Input: User management commands, system configurations
- Output: System reports, activity logs, analytics

##### Payment Gateway ↔ ADLink System:

- Input: Payment confirmations, transaction status
- Output: Payment requests, transaction data

##### Map Services ↔ ADLink System:

- Input: Location data, map tiles
- Output: Location queries, asset positions

#### Notification Services ↔ ADLink System:

- Input: Delivery confirmations
- Output: Push notifications, SMS, emails

#### 4.2.2 Level 1 DFD

The Level 1 DFD decomposes the ADLink system into major functional processes: User Management, Service Discovery, Booking Management, Campaign Management, Payment Processing, Communication, and Analytics.

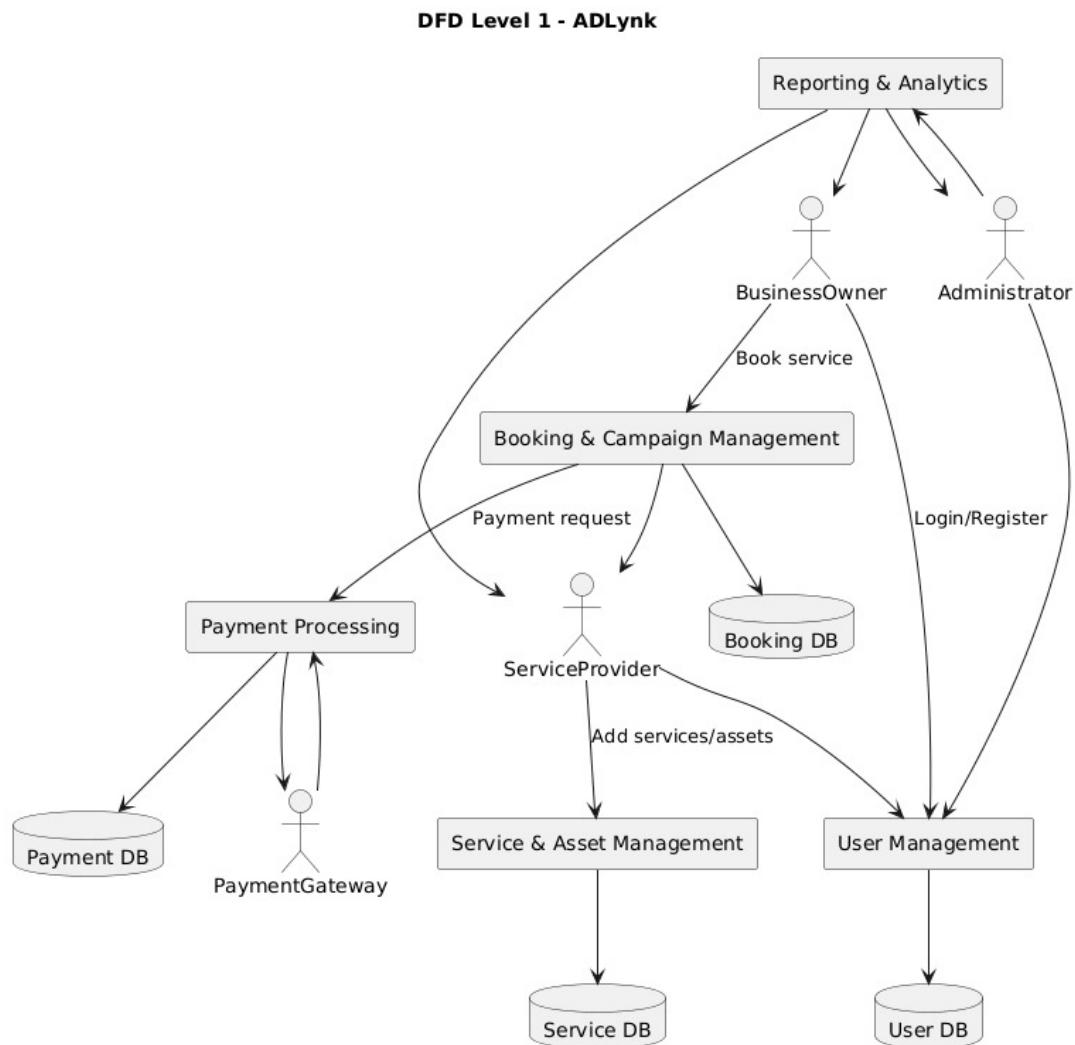


Figure 3: Data Flow Diagram - Level 1

#### Major Processes:

##### 1. User Management Process

- Handles registration, authentication, profile management
- Data stores: User Database, Role Permissions

## 2. Service Discovery Process

- Manages browsing, searching, filtering of services/assets
- Data stores: Service Listings, Asset Inventory

## 3. Booking Management Process

- Processes booking requests, confirmations, cancellations
- Data stores: Bookings Database, Availability Calendar

## 4. Campaign Management Process

- Creates and tracks multi-channel campaigns
- Data stores: Campaign Database, Performance Metrics

## 5. Payment Processing Process

- Handles transactions, invoices, payment history
- Data stores: Transaction Records, Payment History

## 6. Communication Process

- Manages in-app messaging and notifications
- Data stores: Messages Database, Notification Queue

## 7. Analytics and Reporting Process

- Generates reports and performance analytics
- Data stores: Analytics Database, Report Templates

### 4.2.3 Level 2 DFD

The Level 2 DFD provides detailed decomposition of the Booking Management process, showing sub-processes for booking creation, validation, confirmation, and management.

#### **Booking Management Sub-Processes:**

### 1. Validate Booking Request

- Check service/asset availability
- Validate date/time constraints
- Verify user permissions

### 2. Calculate Pricing

- Apply pricing rules
- Calculate total cost
- Apply discounts if applicable

### 3. Process Booking

- Create booking record
- Reserve service/asset
- Update availability calendar

### 4. Generate Confirmation

- Create booking confirmation
- Send notifications to all parties
- Generate invoice

### 5. Manage Booking Status

- Track booking lifecycle
- Handle modifications
- Process cancellations

## 4.3 Sequence Diagram

The Sequence Diagram illustrates the interaction flow for a complete booking process, showing the temporal sequence of messages between the Business Owner, Mobile App, Backend API, Database, Payment Gateway, and Notification Service.

#### Key Interaction Flow:

1. Business Owner browses services
2. App requests service listings from Backend API
3. Backend retrieves data from Database
4. Business Owner selects service and initiates booking
5. App validates booking with Backend
6. Backend checks availability in Database
7. Business Owner confirms and proceeds to payment
8. App sends payment request to Payment Gateway
9. Payment Gateway processes transaction
10. Backend updates booking status in Database
11. Notification Service sends confirmation to all parties
12. App displays booking confirmation to Business Owner

#### Main Activity Flow:

- **Start:** Launch application

- **Decision:** User authenticated?
  - No → Register/Login
  - Yes → Access dashboard
- **Browse Services:** Search and filter available services
- **View Details:** Review service provider profiles and pricing
- **Decision:** Service suitable?
  - No → Continue browsing
  - Yes → Create campaign
- **Configure Campaign:** Set duration, budget, requirements
- **Initiate Booking:** Select services/assets
- **Decision:** Proceed to payment?
  - No → Save draft
  - Yes → Process payment
- **Payment Processing:** Complete transaction
- **Receive Confirmation:** Get booking confirmation
- **Track Campaign:** Monitor performance and analytics
- **Decision:** Campaign complete?
  - No → Continue monitoring
  - Yes → Submit rating and review
- **End**

#### 4.3.1 Service Provider Activity Diagram

This diagram illustrates the workflow for advertising service providers managing their services and bookings.

##### Main Activity Flow:

- **Start:** Login to application
- **Access Dashboard:** View overview of services and bookings
- **Decision:** Action required?
  - **Manage Services:**
    - \* Update service listings
    - \* Modify pricing
    - \* Update availability

- **Handle Bookings:**
  - \* Review new booking requests
  - \* Accept or decline bookings
  - \* Update booking status
- **Communicate:**
  - \* Respond to client messages
  - \* Clarify requirements
- **View Analytics:**
  - \* Check revenue reports
  - \* Review performance metrics
- **Update Status:** Maintain service and booking information
- **End**

#### 4.3.2 Administrator Activity Diagram

This diagram shows the administrative workflow for platform management and monitoring.

##### Main Activity Flow:

- **Start:** Login with admin credentials
- **Access Admin Panel:** View system dashboard
- **Monitor Activity:** Check platform metrics
- **Decision:** Action required?
  - **User Management:**
    - \* Review user accounts
    - \* Verify new registrations
    - \* Handle account issues
  - **Dispute Resolution:**
    - \* Review reported issues
    - \* Communicate with parties
    - \* Make decisions
  - **System Monitoring:**
    - \* Check performance metrics
    - \* Review error logs
    - \* Monitor security alerts
  - **Generate Reports:**
    - \* Create system reports
    - \* Export analytics
    - \* Review trends

- **Take Action:** Implement necessary changes
- **Document Activity:** Log administrative actions
- **End**

## 5 Structural Diagrams

### 5.1 Class Diagram

The Class Diagram depicts the object-oriented structure of the ADLink system, showing the main classes, their attributes, methods, and relationships.

**Key Classes:**

**User (Abstract Base Class)**

- Attributes: userId, email, password, name, phoneNumber, registrationDate
- Methods: register(), login(), updateProfile(), logout()

**BusinessOwner (extends User)**

- Attributes: businessName, businessCategory, advertisingObjectives
- Methods: browseServices(), createCampaign(), bookService(), submitReview()

**ServiceProvider (extends User)**

- Attributes: serviceType, expertise, coverageArea, portfolio
- Methods: listService(), updatePricing(), manageBookings(), viewRevenue()

**AssetOwner (extends User)**

- Attributes: assetType, totalAssets
- Methods: registerAsset(), updateAvailability(), manageBookings()

**Administrator (extends User)**

- Attributes: adminLevel, permissions
- Methods: manageUsers(), resolveDisputes(), generateReports(), configureSystem()

**Service**

- Attributes: serviceId, providerId, serviceType, description, pricing, availability
- Methods: create(), update(), delete(), checkAvailability()

**Asset**

- Attributes: assetId, ownerId, assetType, location, specifications, pricing, availability
- Methods: register(), updateDetails(), checkAvailability(), calculatePrice()

**Campaign**

- Attributes: campaignId, businessOwnerId, campaignName, budget, duration, status, createdAt
- Methods: create(), update(), track(), complete()

**Booking**

- Attributes: bookingId, campaignId, serviceId/assetId, startDate, endDate, totalCost, status
- Methods: create(), confirm(), cancel(), updateStatus()

**Payment**

- Attributes: paymentId, bookingId, amount, paymentMethod, transactionDate, status
- Methods: processPayment(), generateInvoice(), refund()

**Review**

- Attributes: reviewId, bookingId, rating, comment, reviewDate
- Methods: submit(), update(), delete()

**Notification**

- Attributes: notificationId, userId, message, type, sentDate, isRead
- Methods: send(), markAsRead(), delete()

**Relationships:**

- User has associations with multiple classes (one-to-many)
- Campaign has many Bookings (one-to-many)
- Booking has one Payment (one-to-one)
- Service/Asset has many Bookings (one-to-many)
- Booking has one Review (one-to-one)

## 5.2 Component Diagram

The Component Diagram shows the high-level system architecture with major components and their dependencies.

**Main Components:****Mobile Application Layer:**

- UI Components (Flutter Widgets)
- Business Logic Components (ViewModels)
- Local Data Management (Cache/Storage)
- API Client Services

**Backend Services Layer:**

- Authentication Service

- User Management Service
- Service/Asset Management Service
- Booking Service
- Campaign Service
- Payment Service
- Notification Service
- Analytics Service

**Data Layer:**

- Firebase Firestore/Realtime Database
- Firebase Cloud Storage (for media files)
- Local Device Storage (cache)

**External Services:**

- Firebase Authentication
- Payment Gateway API
- Google Maps API
- Firebase Cloud Messaging
- Email/SMS Service

**Component Dependencies:**

- Mobile app depends on Backend Services via REST API
- Backend Services depend on Data Layer for persistence
- Backend Services integrate with External Services for specialized functionality
- All components use Authentication Service for security

## 6 Requirements-Design Traceability

This section maps each functional requirement from the SRS to corresponding design elements, ensuring complete coverage.

| Requirement ID | Requirement Description                         | Design Element(s)                         | Diagram Reference                |
|----------------|-------------------------------------------------|-------------------------------------------|----------------------------------|
| REQ-F-1.1      | Email/password authentication                   | User class, Authentication Service        | Class Diagram, Component Diagram |
| REQ-F-1.2      | User role selection during registration         | User class hierarchy                      | Class Diagram                    |
| REQ-F-1.3      | Service provider and asset owner classification | ServiceProvider and AssetOwner subclasses | Class Diagram                    |
| REQ-F-1.4      | Role-based access control                       | RBAC component, User roles                | Use Case Diagram                 |
| REQ-F-2.1      | User profile management                         | User.updateProfile() method               | Class Diagram                    |
| REQ-F-2.2      | Role-specific profile information               | Extended attributes in role classes       | Class Diagram                    |
| REQ-F-3.1      | Asset listing management                        | Asset class with CRUD methods             | Class Diagram                    |
| REQ-F-3.2      | Asset details                                   | Asset class attributes                    | Class Diagram                    |
| REQ-F-3.3      | Service listing management                      | Service class with CRUD methods           | Class Diagram                    |
| REQ-F-4.1      | Browse advertising services                     | Service Discovery process                 | DFD Level 1                      |
| REQ-F-4.2      | Search and filter functionality                 | Service Discovery process                 | DFD Level 1                      |
| REQ-F-4.3      | View service provider profiles                  | View Details activity                     | Activity Diagram                 |
| REQ-F-5.1      | Digital booking system                          | Booking class, Booking Management         | Class Diagram                    |
| REQ-F-5.2      | Booking customization                           | Booking attributes                        | Class Diagram                    |
| REQ-F-5.3      | Booking tracking                                | Booking.updateStatus() method             | Class Diagram                    |
| REQ-F-6.1      | Campaign management dashboard                   | Campaign class, Campaign Service          | Class Diagram                    |

| Requirement ID | Requirement Description          | Design Element(s)                | Diagram Reference |
|----------------|----------------------------------|----------------------------------|-------------------|
| REQ-F-6.2      | Service provider booking view    | ServiceProvider.manageBookings() | Class Diagram     |
| REQ-F-7.1      | Secure payment processing        | Payment class, Payment Service   | Sequence Diagram  |
| REQ-F-7.2      | Transaction history and invoices | Payment.generateInvoice()        | Class Diagram     |
| REQ-F-7.3      | Payment-campaign association     | Payment references bookingId     | Class Diagram     |
| REQ-F-8.1      | Submit ratings and reviews       | Review class                     | Class Diagram     |
| REQ-F-8.2      | Display ratings on profiles      | Review integration               | Use Case Diagram  |
| REQ-F-9.1      | Administrator management         | Administrator class              | Class Diagram     |
| REQ-F-10.1     | Support requirement changes      | Update methods                   | Class Diagram     |

### Non-Functional Requirements Traceability:

| NFR Category               | Design Approach                                                                                       |
|----------------------------|-------------------------------------------------------------------------------------------------------|
| Performance (REQ-NF-1.x)   | Caching strategy, optimized database queries, lazy loading in UI components                           |
| Security (REQ-NF-2.x)      | HTTPS/TLS for all communication, password hashing, role-based access control, Firebase security rules |
| Reliability (REQ-NF-3.x)   | Error handling in all components, transaction rollback mechanisms, backup strategies                  |
| Usability (REQ-NF-4.x)     | Intuitive UI design in Figma prototype, clear navigation flows in Activity Diagrams                   |
| Availability (REQ-NF-5.x)  | Firebase infrastructure (99.95% uptime SLA), offline capabilities                                     |
| Scalability (REQ-NF-6.x)   | Modular component architecture, cloud-based infrastructure, horizontal scaling capability             |
| Compatibility (REQ-NF-7.x) | Flutter cross-platform framework, responsive UI design                                                |

## 7 Project Resources

### 7.1 GitHub Repository

All project artifacts including diagrams, documentation, source code, and meeting records are maintained in the GitHub repository.

**Repository URL:** [https://github.com/bscs24f48-sys/Software\\_Engineering\\_Project.git](https://github.com/bscs24f48-sys/Software_Engineering_Project.git)

### 7.2 Figma Prototype

Complete interactive mobile application prototype with all user flows is available on Figma.

**Prototype URL:** <https://www.figma.com/design/aLX5Ex3ffKKWs7lJ0iSi0W/Untitled?node-id=0-1&t=WuTiU56FmNsoQlED-1>

### 7.3 Meeting Minutes

Detailed meeting minutes with the Requirement Provider (Saqlain Baig) are maintained in a shared document.

**Meeting Minutes URL:** <https://docs.google.com/spreadsheets/d/1czQSWoIMbhiGJrbsxJiYedit?usp=sharing>

## 8 Meeting Summary

### 8.1 Meeting Overview

Two formal meetings were conducted with the Requirement Provider (Saqlain Baig) during this milestone to review and validate the system design for the ADLink platform.

### 8.2 First Meeting - Design Review

**Date:** January 10, 2026

**Attendees:** Muhammad Hamza, Bilal Ahmed Khan, Saqlain Baig (Requirement Provider)

**Duration:** 90 minutes

#### 8.2.1 Key Discussion Points

- Presented initial system architecture and technology stack selection (Flutter + Firebase)
- Reviewed UML behavioral diagrams (use case, data flow, activity diagrams)
- Discussed role-based access control implementation for four user types
- Reviewed class diagram showing object-oriented structure
- Gathered feedback on multi-channel campaign management approach
- Discussed booking workflow and payment integration strategy

#### 8.2.2 Feedback Received

- Design approach approved with focus on user experience
- Recommended simplifying the booking flow for first-time users
- Suggested adding visual indicators for asset availability on maps
- Requested detailed filtering options for service discovery
- Emphasized importance of secure payment processing
- Recommended including progress indicators for campaign tracking
- Suggested notification preferences for different user roles

#### 8.2.3 Action Items

- Refine booking workflow in activity diagrams based on feedback
- Enhance service discovery filters in sequence diagram
- Create detailed mobile UI mockups in Figma prototype
- Add map integration details to component diagram

- Document notification strategy for each user role
- Prepare interactive prototype demonstration for next meeting

## 8.3 Second Meeting - Prototype Demonstration

**Date:** January 15, 2026

**Attendees:** Muhammad Hamza, Bilal Ahmed Khan, Saqlain Baig (Requirement Provider)

**Duration:** 120 minutes

### 8.3.1 Key Discussion Points

- Demonstrated complete interactive Figma prototype for all four user roles
- Walked through complete user journeys: registration → browsing → booking → payment → tracking
- Showed responsive design considerations for different device sizes
- Presented final UML diagrams with incorporated feedback
- Reviewed requirements-design traceability matrix
- Discussed implementation timeline and technology stack

### 8.3.2 Feedback Received

- Interactive prototype approved for all user roles
- Mobile UI design meets expectations for modern advertising platform
- Navigation flow is intuitive and user-friendly
- Booking process is streamlined and clear
- Payment integration approach is secure and appropriate
- Campaign management dashboard provides necessary insights
- Color scheme and branding suitable for target audience
- All major functional requirements adequately covered in design
- Role-based features properly segregated

### 8.3.3 Approval Status

- **System architecture APPROVED** by Requirement Provider
- **UML diagrams VALIDATED** for all user workflows
- **Prototype APPROVED** for implementation
- **Requirements traceability VERIFIED** - all functional requirements mapped to design

- **Design ready to proceed** to implementation phase
- Minor UI refinements permitted during development based on usability testing

## 8.4 Design Validation

The system design has been comprehensively validated through:

- **Technical Review:** All UML diagrams reviewed with development team for feasibility
- **Stakeholder Review:** Two formal meetings with Requirement Provider (Saqlain Baig)
- **Prototype Testing:** Interactive prototype demonstrated and validated
- **Traceability Verification:** Complete mapping from SRS requirements to design elements confirmed
- **Technology Validation:** Technology stack (Flutter, Firebase) confirmed as appropriate
- **Security Review:** Security constraints and authentication approach validated
- **Performance Analysis:** Performance requirements analyzed against proposed architecture
- **Stakeholder Approval:** Final design approval obtained from Requirement Provider

## 9 Conclusion

This System Design Document provides a comprehensive blueprint for implementing the ADLink Multi-Channel Advertising Management Platform. The design has been carefully crafted to:

- **Meet all requirements** specified in the approved SRS document (Version 1.0)
- **Follow industry-standard practices** using UML notation and mobile design patterns
- **Ensure usability** through validated interactive prototypes
- **Maintain complete traceability** from functional requirements to design elements
- **Address all constraints** including platform, performance, security, and business limitations
- **Support scalability** for nationwide expansion across Pakistan
- **Enable efficient implementation** with clear architectural decisions and component organization

The design adopts a modern mobile-first architecture using Flutter for cross-platform development and Firebase for backend services, ensuring rapid development while maintaining code quality and user experience. The role-based access control system properly segregates functionality for Business Owners, Service Providers, Asset Owners, and Administrators.

All behavioral diagrams (use case, data flow, sequence, and activity diagrams) clearly illustrate system dynamics and user workflows across different scenarios. The structural diagrams (class and component diagrams) define a robust object-oriented architecture that supports modularity, maintainability, and future extensibility.

The comprehensive requirements-design traceability matrix ensures that every functional requirement from the SRS has been addressed in the design, with clear mapping to specific design elements and diagrams. Non-functional requirements including performance, security, reliability, usability, availability, scalability, and compatibility have been thoroughly considered and addressed through appropriate design decisions.

With stakeholder approval obtained and all design artifacts validated, the development team is well-prepared to proceed with system implementation in the next phase. The interactive Figma prototype serves as a detailed specification for the user interface, while the UML diagrams provide clear guidance for backend architecture and data modeling.

### **Document Prepared by:**

Muhammad Hamza (NUM-BSCS-2024-48)

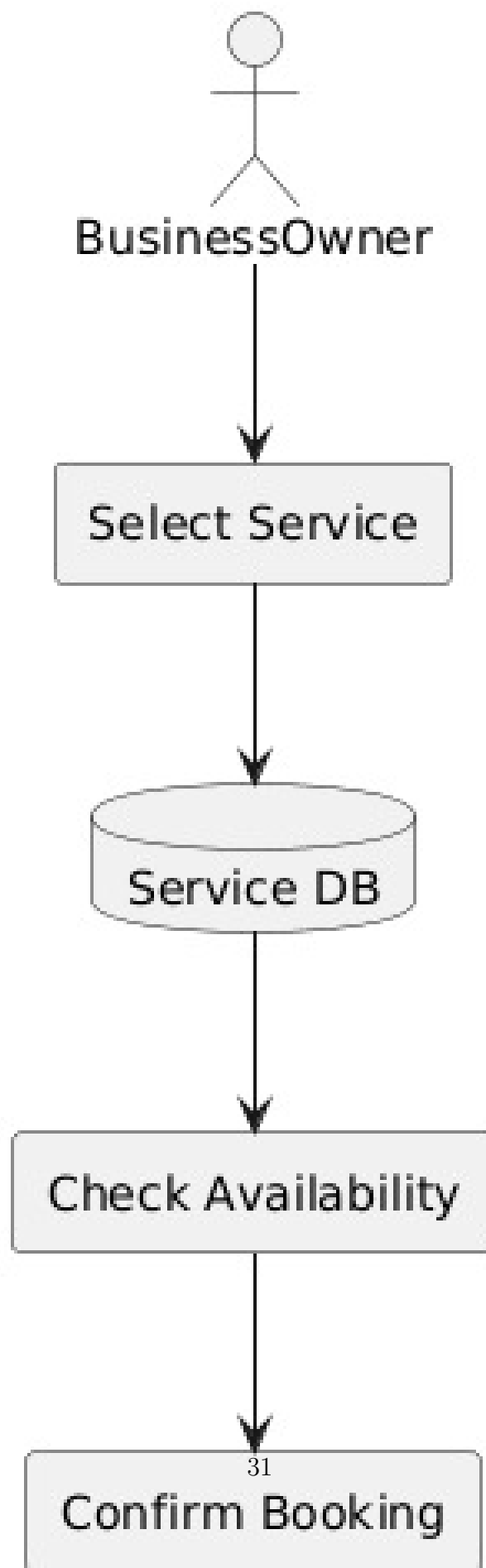
Bilal Ahmed Khan (NUM-BSCS-2024-21)

Department of Computer Science

Namal University, Mianwali

**Date:** January 18, 2026

## DFD Level 2 - Booking Process



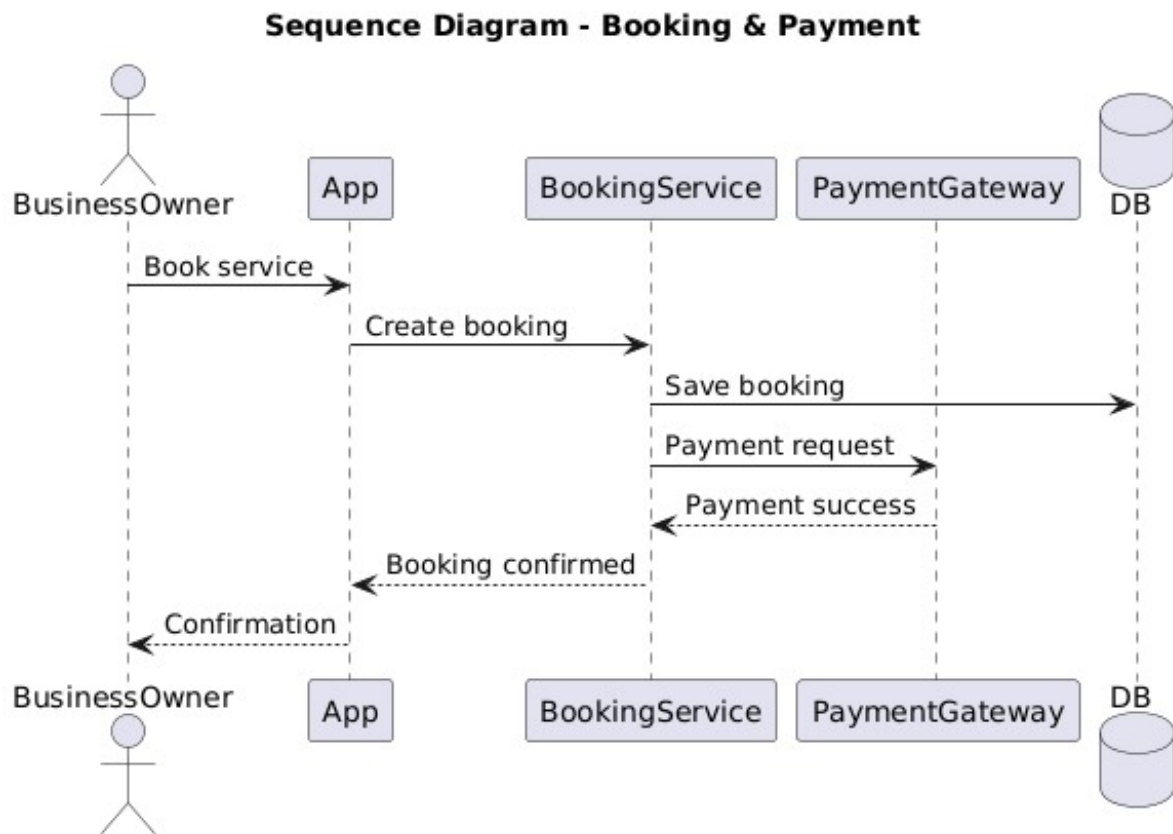


Figure 5: Sequence Diagram

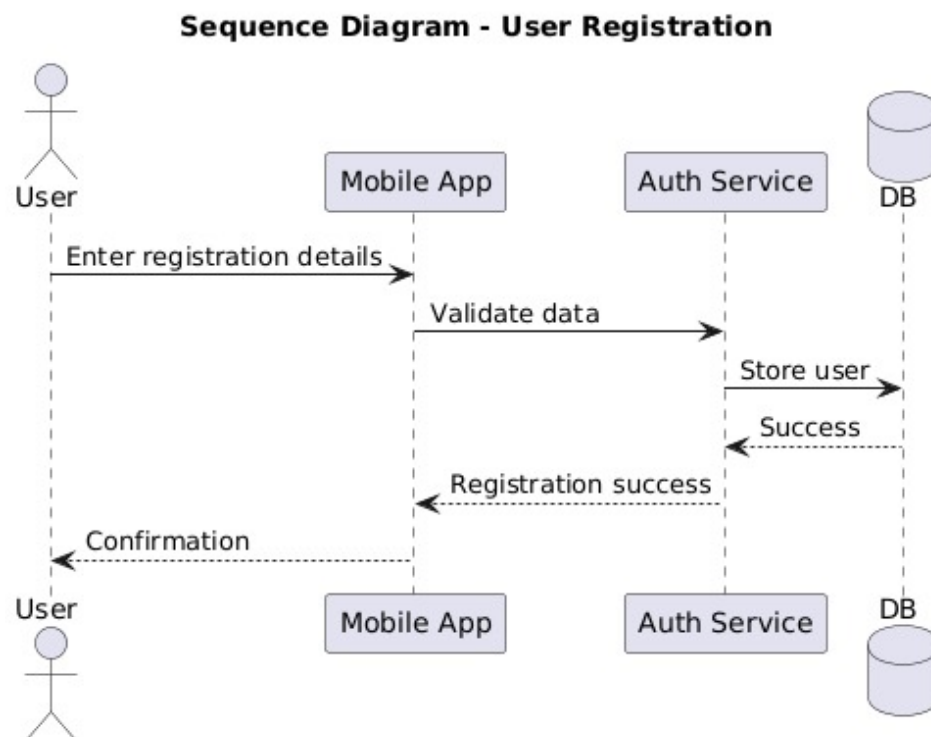


Figure 6: Activity Diagram - Business Owner

### Activity Diagram - Campaign Booking

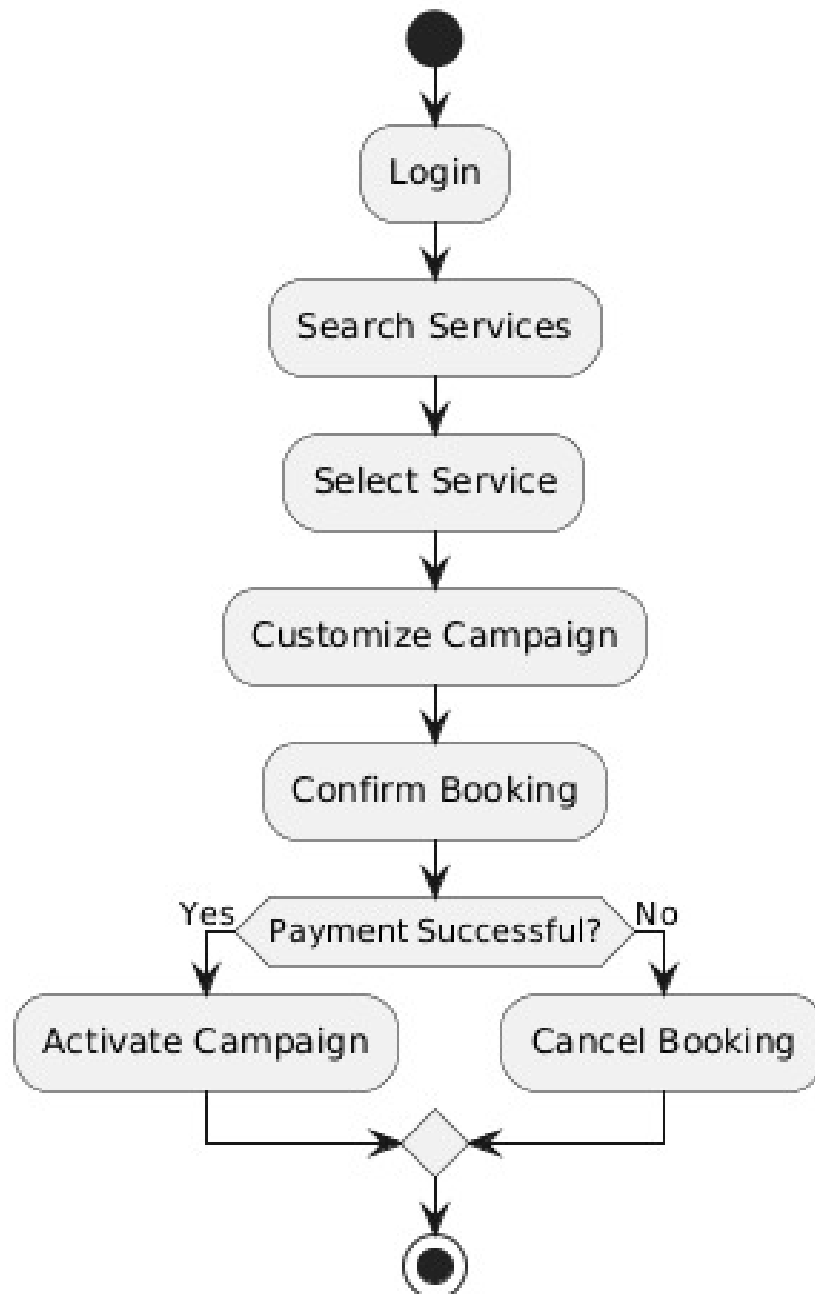


Figure 7: Activity Diagram - Administrator

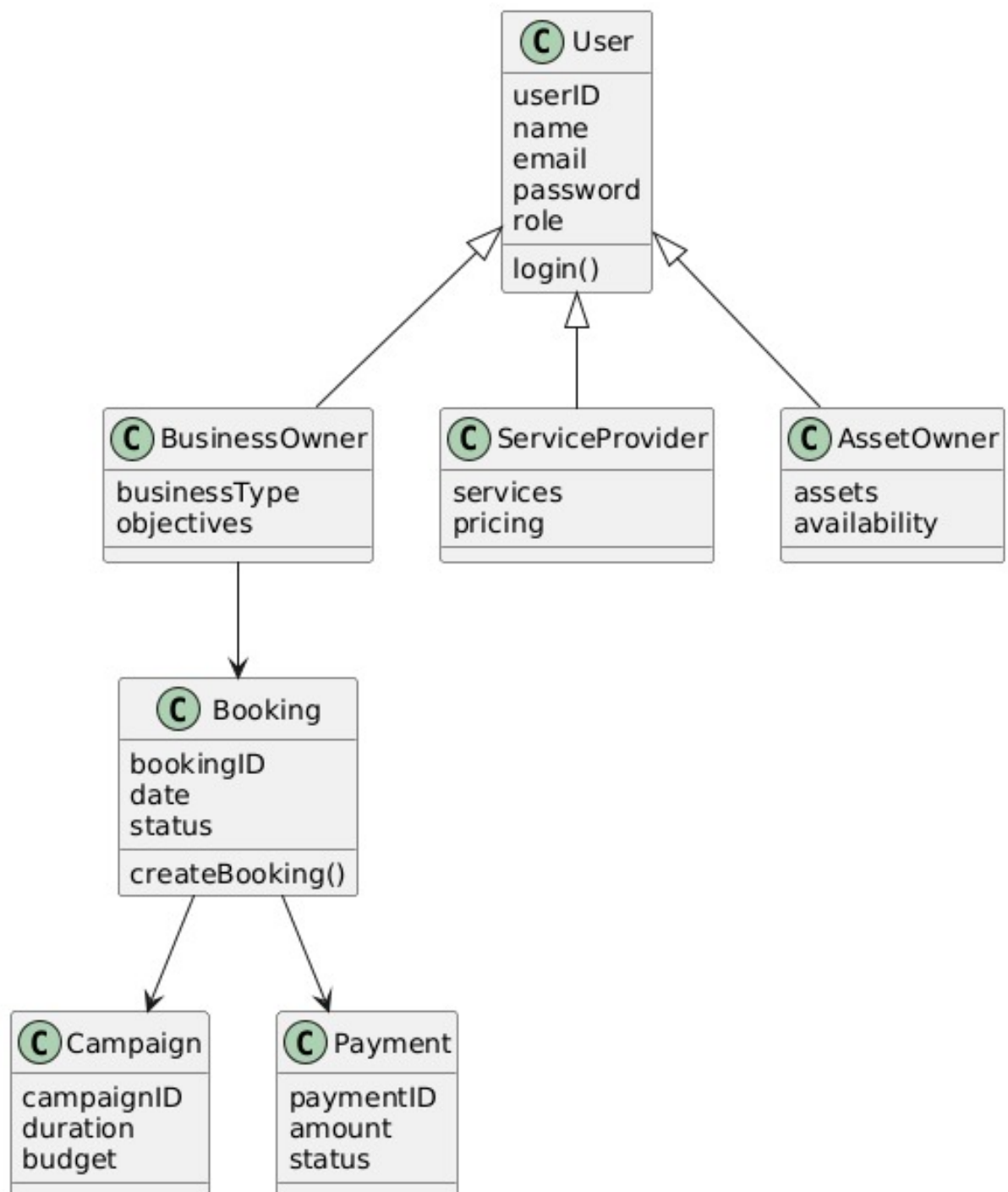
**Class Diagram - ADLynk**

Figure 8: Class Diagram

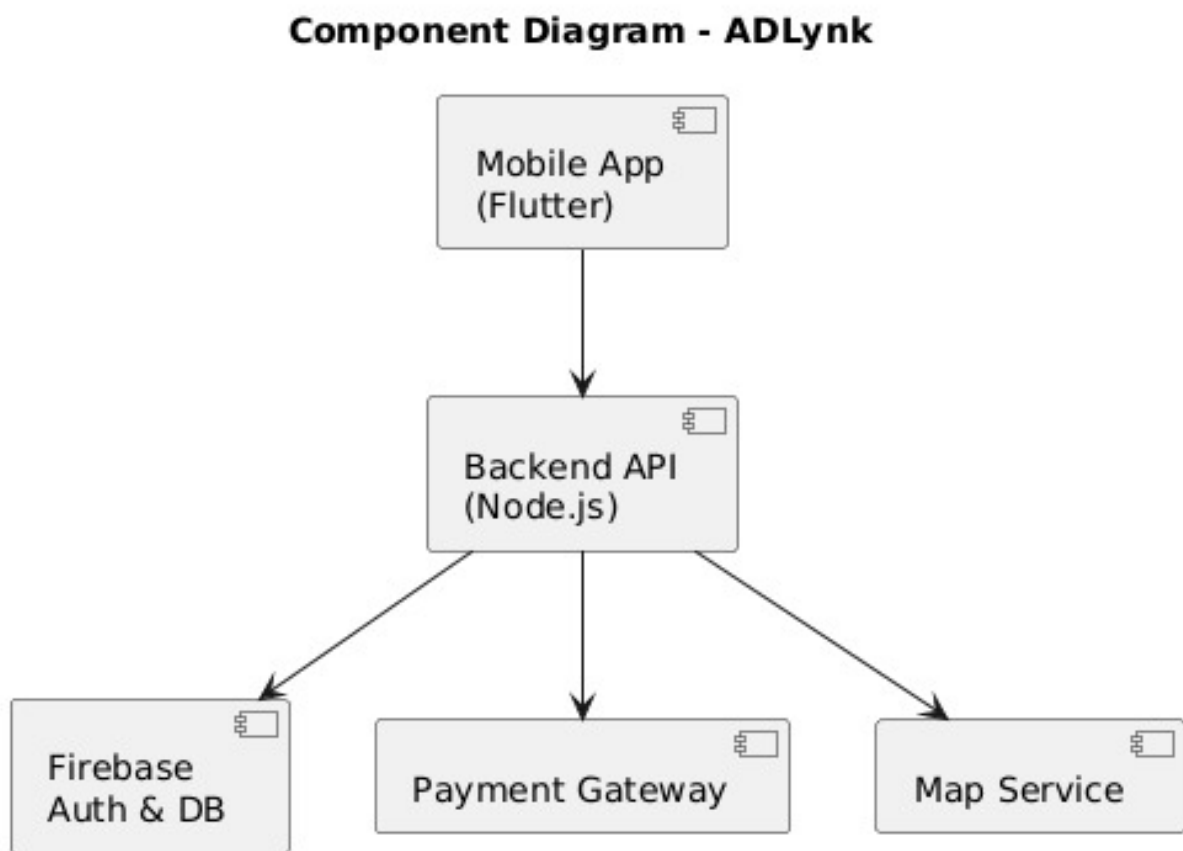


Figure 9: Component Diagram